Enterprise ML Platform

Владимир Селифанов

Table of Contents

Слайд 1: Enterprise ML Platform . . . 1
- Курсовая работа по System Design . . . 1

Слайд 2: Проблема и цель . . . 2
- Контекст . . . 2
- Ключевые вызовы . . . 2
- Цель . . . 2

Слайд 3: Ключевые требования . . . 3
- Функциональные требования (Must Have) . . . 3
- Нефункциональные требования (измеримые) . . . 3

Слайд 4: Концептуальная архитектура . . . 4
- Пользователи и интеграции . . . 4
- Архитектурный стиль . . . 4

Слайд 5: Архитектура решения . . . 5
- Ключевые паттерны . . . 5
- HA-топология . . . 6

Слайд 6: Выбор технологий . . . 7

Слайд 7: Хранение данных . . . 8
- Стратегия хранения и CAP-позиция . . . 8
- CQRS на двух уровнях . . . 8
- Рост данных (прогноз) . . . 8

Слайд 8: Безопасность и Observability . . . 9
- Безопасность: Defence in Depth . . . 9
- Observability Stack . . . 9

Слайд 9: Доставка решения . . . 11
- CI/CD Pipeline . . . 13
- Инфраструктура как код . . . 13
- Тестирование . . . 13

Слайд 10: Риски и компромиссы . . . 14
- Топ-5 рисков . . . 14
- Ключевые trade-off . . . 14

Слайд 1: Enterprise ML Platform

Курсовая работа по System Design

Проект	Enterprise ML Platform
Курс	OTUS System Design
Автор	Владимир Селифанов
Дата	2026
Объём	24 секции · 16 диаграмм · 6 ДЗ

Слайд 2: Проблема и цель

Контекст

Крупные компании развёртывают десятки ML-моделей в production. Без единой платформы каждая команда решает одни и те же проблемы: версионирование, безопасное развёртывание, мониторинг качества, изоляция ресурсов.

Ключевые вызовы

- **Governance** — контроль жизненного цикла модели от Draft до Deprecated
- **Scale** — 100+ моделей, 50+ тенантов, 10 000 RPS пиковой нагрузки
- **Safety** — недопустимо развернуть некачественную модель в production без согласований
- **Multi-tenancy** — изоляция данных и ресурсов между командами/проектами
- **Observability** — детекция дрифта модели, SLO для каждого инференс-эндпоинта

Цель

Единая корпоративная платформа, которая обеспечивает:

1. Управляемый жизненный цикл ML-моделей с workflow согласований
2. Высокопроизводительный инференс (sync / async / streaming / batch)
3. Мультитенантность с изоляцией данных и resource quotas
4. Полный стек observability и безопасности класса Enterprise

Слайд 3: Ключевые требования

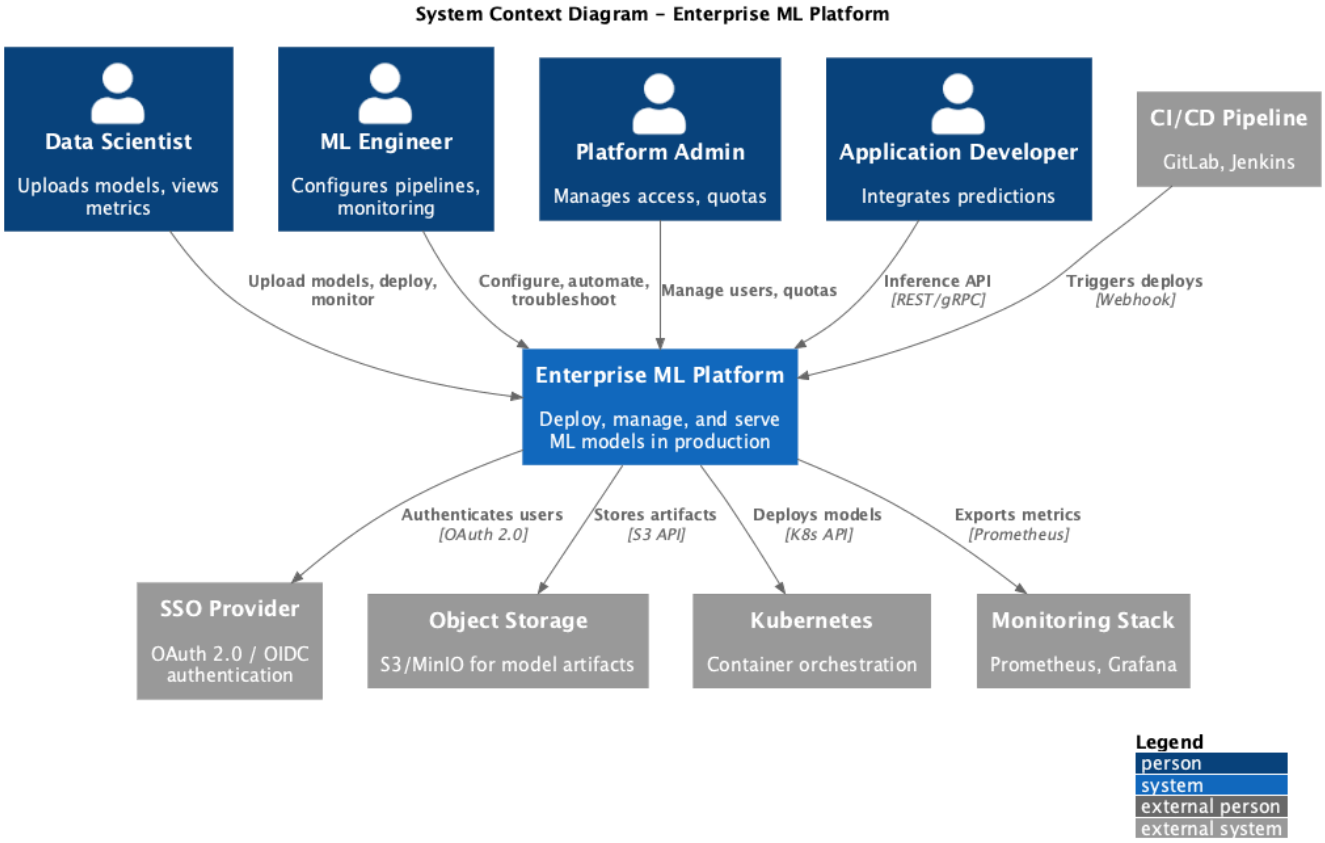
Функциональные требования (Must Have)

Домен	Ключевые FR
Core ML Serving	Sync p99 ≤100ms · Async callback · LLM streaming · A/B testing · Canary deploy
Model Lifecycle	Draft → Validating → Staging → Production → Deprecated · Workflow Risk/Compliance Review
Batch Inference	1M+ записей/задача · Lifecycle Submitted→Running→Completed · DLQ · Retry
Multi-tenancy	Изоляция данных · Квоты и rate limiting · RBAC per tenant
Billing	Учёт compute/requests/storage · Бюджетные алерты (50% / 80% / 100%)
Security	API Keys + OAuth 2.0 / SSO · RBAC · Audit log · Шифрование AES-256 / TLS

Нефункциональные требования (измеримые)

ID	Требование	Значение
NFR-101	Sync inference latency	p50 ≤30ms · p99 ≤100ms
NFR-201	Availability Critical компоненты	SLA 99.9%
NFR-205	Region failover	RTO ≤5 мин
NFR-301	Масштабируемость инференса	до 100+ инстансов
NFR-304	Время реакции автомасштабирования	≤2 мин
NFR-501	Хранение артефактов	10 TB (1 TB/тенант Enterprise)

Слайд 4: Концептуальная архитектура



Пользователи и интеграции

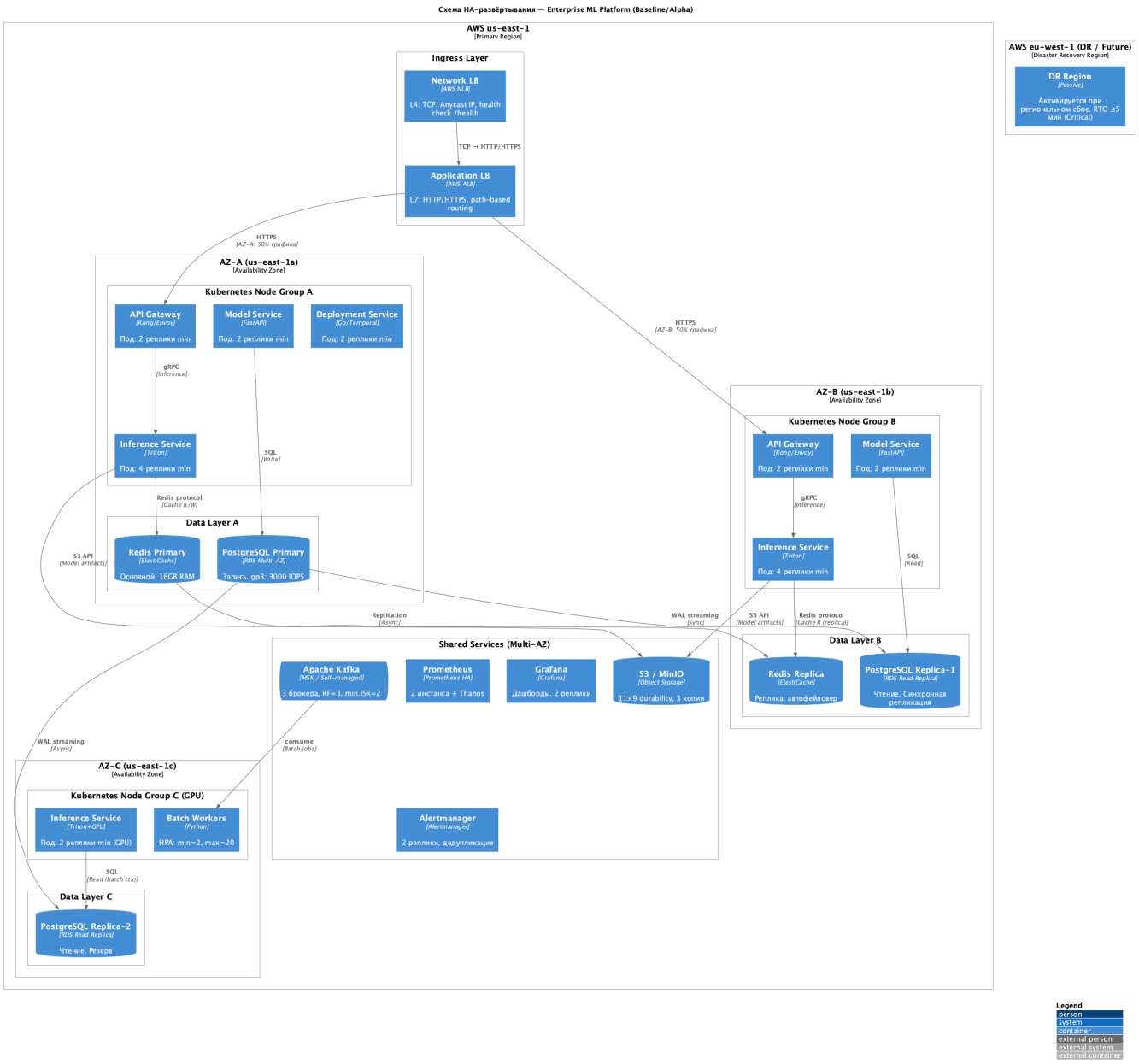
Актор	Взаимодействие
Data Scientist	REST API / CLI / SDK (Python) — публикация и управление моделями
ML Engineer / MLOps	Deployment Service — пайплайны CI/CD, health checks, rollback
Enterprise Tenant	REST + gRPC — инференс запросы, batch jobs, BYOB S3
External IdP	OAuth 2.0 / OIDC — SSO для пользователей платформы
Monitoring System	Alertmanager → PagerDuty / мессенджер команды

Архитектурный стиль

Микросервисы + **Event-Driven** — ключевые обоснования:

- Inference Service масштабируется независимо от Model Service
- Сбой Deployment Service не влияет на работающий инференс
- Kafka обеспечивает развязку компонентов и обработку пиков нагрузки

Слайд 5: Архитектура решения



Ключевые паттерны

Паттерн	Применение
State Machine	Жизненный цикл модели: Draft → Validating → Staging → Production → Deprecated
CQRS	Redis (inference hot path, hit rate >80%) + PG read replica (UI/reporting)
Saga (Orchestration)	Транзакционный контроль квот через Temporal; компенсация при сбоях
Circuit Breaker	Inference Service → GPU Workers; предотвращение каскадных отказов

Паттерн	Применение
Canary / Blue-Green	Безопасное развёртывание: постепенный сдвиг трафика + мгновенный откат
CDC (Debezium)	PostgreSQL WAL → Kafka: decoupling Model Service и Deployment Service
Scale-to-zero	KEDA по Kafka lag: неактивные модели (0 RPS за 30 мин) → 0 реплик
Rate Limiting	Per-tenant лимиты; burst 2× на первые 30s; DLQ для rejected requests

HA-топология

3 Availability Zones · Active-Active · RTO <2 мин при AZ failure · RPO=0 (sync PG replica)

Слайд 6: Выбор технологий

Компонент	Технология	Обоснование
API Gateway	Kong / Envoy	Rate limiting, auth, TLS termination — единая точка входа; plugin ecosystem Kong
Inference Service	Triton Inference Server	NVIDIA-оптимизированный serving: PyTorch, TensorFlow, ONNX из коробки; dynamic batching
Deployment Orchestration	Go + Temporal	Saga workflows с гарантированным выполнением; durable timers; retry/compensation built-in
Model Service	Python + FastAPI	Основная аудитория — Python DS/MLE; async handlers; Pydantic validation
Event Bus	Apache Kafka (MSK)	RF=3, min.ISR=2; partitions по tenant_id; retention 7 дней; CDC-ready (Debezium)
Secrets Management	HashiCorp Vault	Dynamic secrets (TTL 1h для DB creds); PKI engine; cloud-agnostic; audit log
Autoscaling	KEDA + HPA	Event-driven scaling по Kafka lag (batch workers) + CPU/RPS (inference)
IaC	Terraform + Helm	Terraform — AWS ресурсы; Helm — K8s объекты; чёткое разделение ответственности

Слайд 7: Хранение данных

Стратегия хранения и CAP-позиция

Данные	Хранилище	CAP	Обоснование
Метаданные моделей	PostgreSQL (Multi-AZ)	**CP**	ACID-транзакции; sync replica RPO=0; sources of truth
Артефакты моделей	S3 / MinIO	**AP**	Strong consistency для новых PUT; 11 девяток durability; lifecycle to Glacier
Кэш предсказаний	Redis (Sentinel)	**AP**	Sub-ms latency; TTL; не источник истины — восстанавливается из PG
События / Аудит	Kafka → PostgreSQL	**AP**	Decoupling; replay; CDC-ready; retention 7 дней
Метрики	VictoriaMetrics	**AP**	10:1 сжатие vs Prometheus; горизонтальное масштабирование; PromQL совместим
Batch Data	S3 (BYOB / Managed)	**AP**	Enterprise: cross-account IAM; self-service: managed buckets с tenant prefix

CQRS на двух уровнях

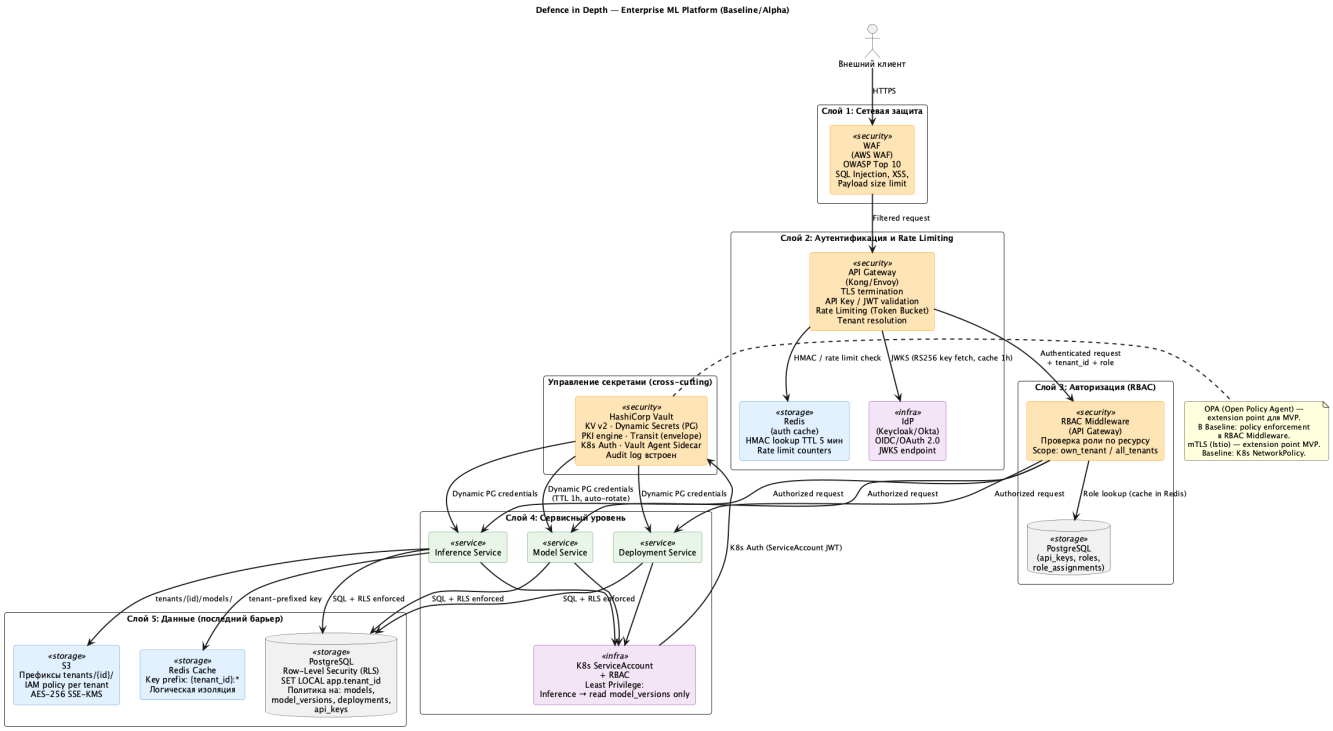
- **Уровень 1 (hot path)**: Inference → Redis hit (>80%) → ответ без PG
- **Уровень 2 (model registry)**: Write → PG primary; Read UI/API → PG async replica (AZ-C)

Рост данных (прогноз)

Компонент	Год 1	Год 3	Год 5
Артефакты S3	1 TB	5 TB	15 TB
Метаданные PG	10 GB	40 GB	100 GB
VictoriaMetrics	50 GB/мес	150 GB/мес	350 GB/мес

Слайд 8: Безопасность и Observability

Безопасность: Defence in Depth



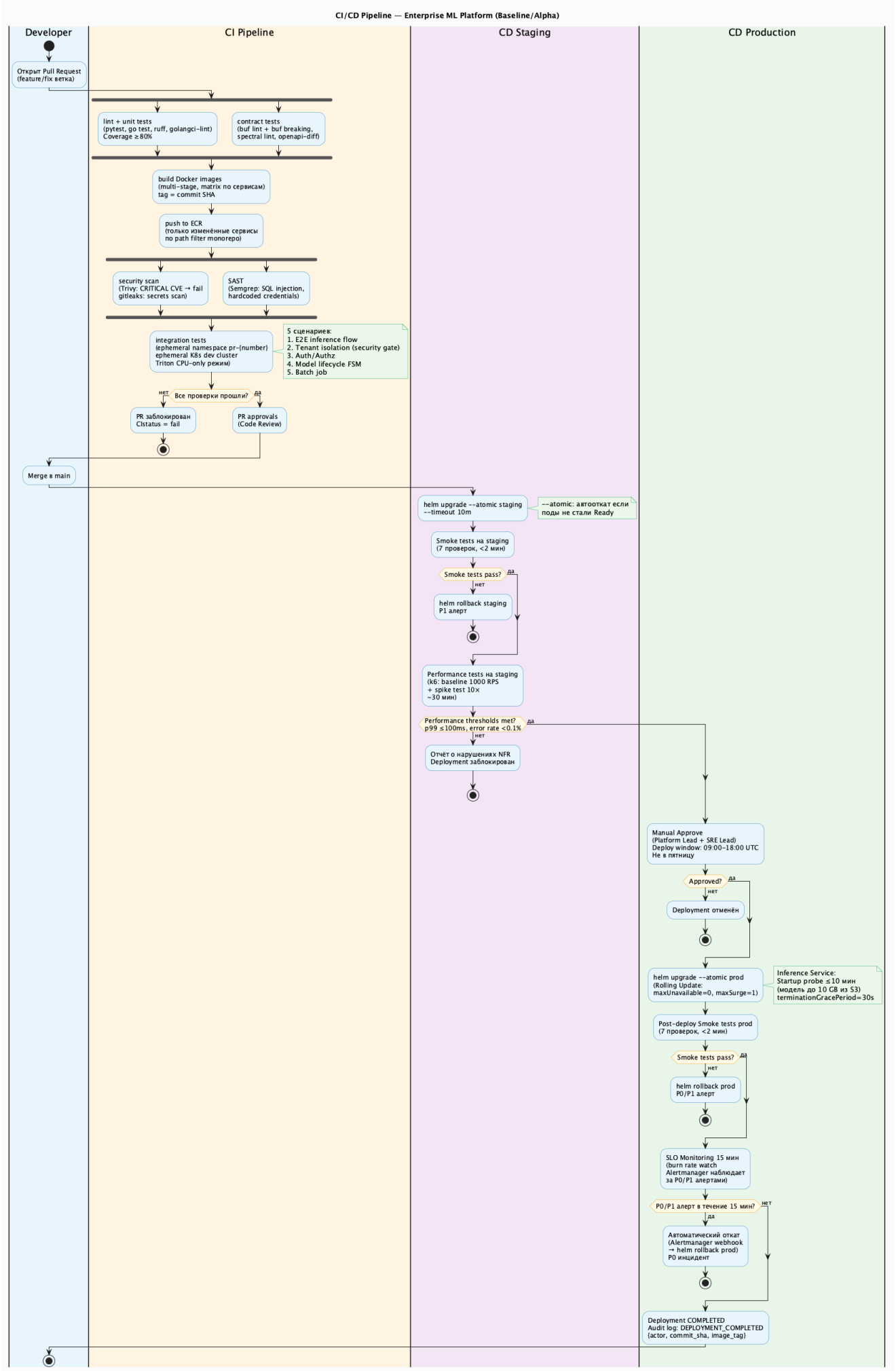
Слой	Решение
Аутентификация	API Keys (HMAC-SHA256, prefix `mlp_`, secret scanning) · OAuth 2.0/OIDC (Keycloak/Okta)
Авторизация	RBAC в API Gateway middleware · PostgreSQL Row-Level Security как последний барьер
Secrets	HashiCorp Vault: dynamic DB credentials (TTL 1h) · PKI engine · Vault Agent Sidecar
Шифрование	TLS 1.3 (external) · mTLS межсервисный (MVP) · AES-256 SSE-KMS (S3 артефакты)
Audit	Аудит-лог всех действий ≥1 год · CloudTrail S3 access · Kafka audit топик

Observability Stack

Уровень	Компонент	Роль
Сбор	Prometheus Agent · Fluent Bit	Метрики pull 15s · Логи из K8s stdout
Хранение	VictoriaMetrics · Loki	TSDB 90d детально / 1y агрегат · Логи S3-backed 1 год

Уровень	Компонент	Роль
Визуализация	Grafana · Alertmanager	PromQL/LogQL/TraceQL · P0/P1 → PagerDuty · P2/P3 → мессенджер
ML Quality	Drift Detector	Мониторинг распределений input/output; алерт при дрифте → P2

Слайд 9: Доставка решения


CI/CD Pipeline — Enterprise ML Platform (Baseline/Alpha)
Developer
CI Pipeline
CD Staging
CD Production
Открыт Pull Request (feature/fix ветка)
lint + unit tests (pytest, go test, ruff, golangci-lint) Coverage ≥80%
contract tests (buf lint + buf breaking, spectral lint, openapi-diff)
build Docker images (multi-stage, matrix по сервисам) tag = commit SHA
push to ECR (только изменённые сервисы по path filter monorepo)
security scan (Trivy: CRITICAL CVE → fail gitleaks: secrets scan)
SAST (Semgrep: SQL injection, hardcoded credentials)
integration tests (ephemeral namespace pr-{number} ephemeral K8s dev cluster Triton CPU-only режим)
5 сценариев: 1. E2E inference flow 2. Tenant isolation (security gate) 3. Auth/Authz 4. Model lifecycle FSM 5. Batch job
Все проверки прошли?
нет
да
PR заблокирован CIstatus = fail
PR approvals (Code Review)
Merge в main
helm upgrade --atomic staging --timeout 10m
--atomic: автооткат если поды не стали Ready
Smoke tests на staging (7 проверок, <2 мин)
Smoke tests pass?
да
нет
helm rollback staging P1 алерт
Performance tests на staging (k6: baseline 1000 RPS + spike test 10× ~30 мин)
Performance thresholds met? p99 ≤100ms, error rate <0.1%
да
нет
Отчёт о нарушениях NFR Deployment заблокирован
Manual Approve (Platform Lead + SRE Lead) Deploy window: 09:00–18:00 UTC Не в пятницу
Approved?
да
нет
Deployment отменён
helm upgrade --atomic prod (Rolling Update: maxUnavailable=0, maxSurge=1)
Inference Service: Startup probe ≤10 мин (модель до 10 GB из S3) terminationGracePeriod=30s
Post-deploy Smoke tests prod (7 проверок, <2 мин)
Smoke tests pass?
да
нет
helm rollback prod P0/P1 алерт
SLO Monitoring 15 мин (burn rate watch Alertmanager наблюдает за P0/P1 алертами)
P0/P1 алерт в течение 15 мин?
нет
да
Автоматический откат (Alertmanager webhook → helm rollback prod) P0 инцидент
Deployment COMPLETED Audit log: DEPLOYMENT_COMPLETED {actor, commit_sha, image_tag}

CI/CD Pipeline

Этап	Содержание
Build	Docker image build → unit tests → SAST (Bandit/Semgrep) → image push to ECR
Test	Integration tests (Testcontainers) · Tenant isolation tests · Contract tests (Pact)
Staging Deploy	Helm upgrade staging · Smoke tests · Performance baseline (k6, NFR-101 p99 ≤100ms)
Production Deploy	Canary 5% → 25% → 100% · Health check gate · Auto-rollback при error rate >1%
IaC Apply	terraform plan → review → apply (environments: dev / staging / prod)

Инфраструктура как код

Инструмент	Область
Terraform	AWS VPC, EKS, RDS Multi-AZ, MSK, ElastiCache, S3, IAM, Vault on K8s
Helm	K8s Deployments, HPA, KEDA ScaledObjects, NetworkPolicies, ServiceAccounts

Тестирование

- **Unit** — бизнес-логика сервисов (pytest / go test)
- **Integration** — Testcontainers: реальные PG + Redis + Kafka
- **Load** — k6: 10 000 RPS, проверка NFR-101 p99 ≤100ms
- **Security** — tenant isolation автотесты в каждом PR

Слайд 10: Риски и компромиссы

Топ-5 рисков

ID	Риск	Вероят.	Влияние	Митигация
R-01	Нехватка GPU при всплесках	Средняя	Критическое	Reserved capacity · CPU fallback · scale-to-zero для неактивных
R-02	Необнаруженный дрифт модели	Высокая	Значительное	Drift Detector → алерт P2 · быстрый откат одним действием
R-04	Сбой развёртывания в production	Средняя	Критическое	Canary + health check gate · auto-rollback при error >1%
R-05	Нарушение безопасности	Низкая	Критическое	RBAC + RLS + Vault · tenant isolation tests в CI · AES-256
R-15	Noisy neighbor среди тенантов	Средняя	Значительное	Per-tenant rate limits · K8s ResourceQuota · burst 2× только 30s

Ключевые trade-off

Решение	Выигрыш	Компромисс
Микросервисы vs монолит	Независимое масштабирование, изоляция отказов	Операционная сложность, сетевые накладные расходы
PostgreSQL CP vs eventual	RPO=0, ACID-транзакции, источник истины	Вертикальный bottleneck при >50K RPS
Redis кэш (AP)	Sub-ms inference hit path, >80% cache hit	Split-brain допустим — Redis не источник истины
Шардирование PG отложено	Нет избыточной сложности на Baseline	Структурный рефакторинг при >50K RPS sustained
Baseline: explicit Kafka publish (не CDC)	Меньше операционной сложности (нет Debezium)	При росте consumers — coupling растёт; CDC — MVP extension point